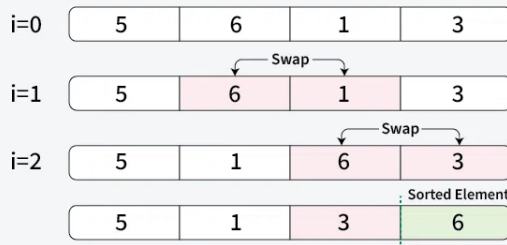


01
Step

Placing the 1st largest element at its correct position

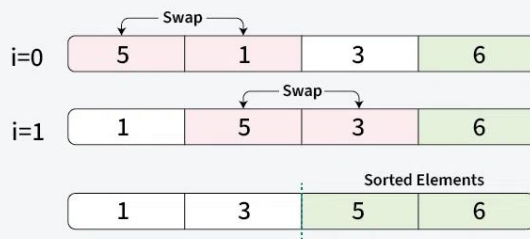


Bubble sort

Figure 1 (geeksforgeeks.org)

02
Step

Placing 2nd largest element at its correct position



Bubble sort

Figure 2 (geeksforgeeks.org)

03
Step

Placing 3rd largest element at its correct position



Bubble sort

Figure 3 (geeksforgeeks.org)

Bubble Sort

From my understanding of bubble sort, it's a sorting algorithm that swaps elements of an array starting with finding the largest element in the array and comparing it to the next element in the array and so on.

It takes multiple passes to complete as it compares one value to each element in the array in each pass before moving to the next. Figures 1-3 show this example. While the algorithm is good for smaller arrays, it is less ideal for larger arrays as shown on the next page

Example of Bubble sort code

```
Microsoft Visual Studio Debug Console
Example array list using bubble sort:

Unsorted array: 64 34 25 12 22 11 90

Array after pass 1: 34 25 12 22 11 64 90
Array after pass 2: 25 12 22 11 34 64 90
Array after pass 3: 12 22 11 25 34 64 90
Array after pass 4: 12 11 22 25 34 64 90
Array after pass 5: 11 12 22 25 34 64 90
Array after pass 6: 11 12 22 25 34 64 90

Sorted array: 11 12 22 25 34 64 90
```

In this example output, bubble sort takes the array and after 6 passes sorts the array in the correct position.

```
Unsorted array with 10000 elements:
41 8467 6334 6500 9169 5724 1478 9358 6962 4464
Sorted array with 10000 elements:
0 0 2 3 3 3 4 4 6 8
Unsorted array with 20000 elements:
19484 2693 17194 10605 17059 6838 556 15179 18296 10782
Sorted array with 20000 elements:
0 0 1 2 2 2 6 6 7 7
Unsorted array with 30000 elements:
8970 8320 14252 20912 1360 6954 10443 11838 23376 652
Sorted array with 30000 elements:
1 1 3 3 3 4 4 5 5 6
Unsorted array with 40000 elements:
3154 19830 8543 16859 9519 8328 9072 19727 17818 5747
Sorted array with 40000 elements:
0 1 1 1 1 3 4 4 4 5

Bubble sort time:
for an array of 10000: 615 milliseconds
for an array of 20000: 2475 milliseconds
for an array of 30000: 6020 milliseconds
for an array of 40000: 9407 milliseconds
Name: Adam Lee
JagNumber: J00762612
```

This photo shows an example for using multiple arrays from 10,000 to 40,000 and shows the time it takes to sort them.

Code for the bubble sort

```
Test_File.cpp  x
Test_File  (Global Scope)  main()

1  /* bubble sort program that reads 10000 to 40000 elements and sorts them using bubble sort
2  * credited - chegg.com
3  * credited - geeksforgeeks.org
4  * credited - stackoverflow.com
5  * modified by Adam Lee, JagNumber: J00762612
6  * date: 10/31/2024
7  */
8  #include <iostream>
9  #include <stdio.h>
10 #include <chrono> // for timer
11 #include <cstdlib> // for rand()
12
13 using namespace std::chrono;
14
15 // swap elements of array
16 void swap(int* a, int* b)
17 {
18     int temp = *a;
19     *a = *b;
20     *b = temp;
21 }
22
23 // bubble sort function
24 void bubbleSort(int array[], int len, bool printPasses = false)
25 {
26     int i, j; // integer values i and j respectfully
27     bool swapped; // boolean for swapped function
28     for (i = 0; i < len - 1; i++) // for loop that compares elements in the array, returns false if smaller
29     {
30         swapped = false;
31         for (j = 0; j < len - i - 1; j++) // for loop that compares the next largest and swaps if need
32         {
```

```
Test_File.cpp  x
Test_File  (Global Scope)  main()

32         {
33             if (array[j] > array[j + 1]) // if statement that swaps the elements in the array.
34             {
35                 swap(&array[j], &array[j + 1]);
36                 swapped = true;
37             }
38         }
39
40         // Print the array after each complete pass if specified
41         if (printPasses) {
42             printf("Array after pass %d: ", i + 1);
43             for (int k = 0; k < len; k++) {
44                 printf("%d ", array[k]);
45             }
46             printf("\n");
47         }
48
49         if (!swapped) // if no swaps, break from loop
50             break;
51     }
52 }
53
54 // function to display the array
55 void display(int array[], int size)
56 {
57     for (int i = 0; i < size; i++)
58         printf("%d ", array[i]); // print array
59 }
60
61 int main()
62 {
63     int array[] = { 64, 34, 25, 12, 22, 11, 90 }; // example array
64     int n = sizeof(array) / sizeof(array[0]); // math for length of array
```

```

61 int main()
62 {
63     int array[] = { 64, 34, 25, 12, 22, 11, 90 }; // example array
64     int n = sizeof(array) / sizeof(array[0]); // math for length of array
65     printf("Example array list using bubble sort:\n");
66     printf("\n");
67
68     printf("Unsorted array: ");
69     display(array, n); // Display unsorted array
70     printf("\n\n");
71
72     // Call bubble sort with pass print as true
73     bubbleSort(array, n, true);
74     printf("\nSorted array: ");
75     display(array, n); // Display sorted array
76     printf("\n");
77
78     // Testing with sample random data
79     int array_10000[10000];
80     int array_20000[20000];
81     int array_30000[30000];
82     int array_40000[40000];
83
84     long long time_10000, time_20000, time_30000, time_40000;
85
86     // Starts with 10000 elements
87     auto a = high_resolution_clock::now(); // starts timer for first array
88
89
90     for (int i = 0; i < 10000; i++) // for loop that counts through 0 to 10000 and increments
91         array_10000[i] = rand() % 10000; // gets random number generated
92     printf("\nUnsorted array with 10000 elements: \n"); // displays first 10 elements of the unsorted array.
93     display(array_10000, 10);

```

```

93     display(array_10000, 10);
94
95     printf("\nSorted array with 10000 elements: \n");
96     bubbleSort(array_10000, 10000); // calls bubble sort function for 10000 number array
97     display(array_10000, 10); // display first 10 elements of sorted array
98
99     auto b = high_resolution_clock::now(); // stops timer for first array
100    time_10000 = duration_cast<milliseconds>(b - a).count(); // store time taken
101
102    // Repeat for 20000 elements
103    a = high_resolution_clock::now();
104
105
106    for (int i = 0; i < 20000; i++)
107        array_20000[i] = rand() % 20000;
108    printf("\nUnsorted array with 20000 elements: \n");
109    display(array_20000, 10);
110
111    printf("\nSorted array with 20000 elements: \n");
112    bubbleSort(array_20000, 20000);
113    display(array_20000, 10);
114
115    b = high_resolution_clock::now();
116    time_20000 = duration_cast<milliseconds>(b - a).count();
117
118    // Repeat for 30000 elements
119    a = high_resolution_clock::now();
120
121
122    for (int i = 0; i < 30000; i++)
123        array_30000[i] = rand() % 30000;
124    printf("\nUnsorted array with 30000 elements: \n");
125    display(array_30000, 10);

```

```

126     printf("\nSorted array with 30000 elements: \n");
127     bubbleSort(array_30000, 30000);
128     display(array_30000, 10);
129
130     b = high_resolution_clock::now();
131     time_30000 = duration_cast<milliseconds>(b - a).count();
132
133     // Repeat for 40000 elements
134     a = high_resolution_clock::now();
135
136
137     for (int i = 0; i < 40000; i++)
138         array_40000[i] = rand() % 40000;
139     printf("\nUnsorted array with 40000 elements: \n");
140     display(array_40000, 10);
141
142     printf("\nSorted array with 40000 elements: \n");
143     bubbleSort(array_40000, 40000);
144     display(array_40000, 10);
145
146     b = high_resolution_clock::now();
147     time_40000 = duration_cast<milliseconds>(b - a).count();
148
149     // Print times for each array
150     printf("\n");
151     printf("\nBubble sort time:\n");
152     printf("for an array of 10000: %lld milliseconds\n", time_10000);
153     printf("for an array of 20000: %lld milliseconds\n", time_20000);
154     printf("for an array of 30000: %lld milliseconds\n", time_30000);
155     printf("for an array of 40000: %lld milliseconds\n", time_40000);
156     printf("Name: Adam Lee\nJagNumber: J00762612\n"); // name and jagnumber
157
158     return 0;

```

Quicksort:

From my understanding of the quicksort algorithm is that it's a "divide and conquer" (geeksforgeeks.org) algorithm where it splits up the array into multiple pieces and sort them that way, or "halves" (geeksforgeeks.org). It creates a pivot, which can be any element, but in the best case, using the last element is ideal. From there it takes the pivot element and sorts them based on that element, *figure 4* shows best. While it might not be ideal for smaller array as it could be

longer in some cases, it's best to use this algorithm when dealing with larger arrays. An example of this is shown on the next page.

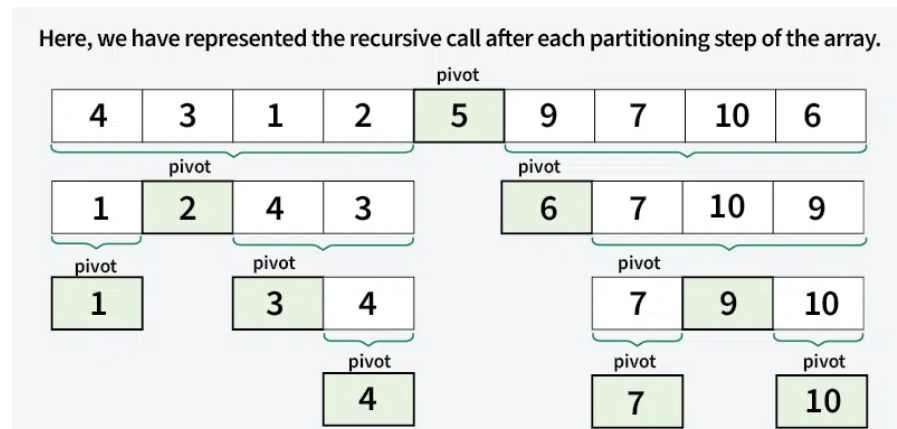
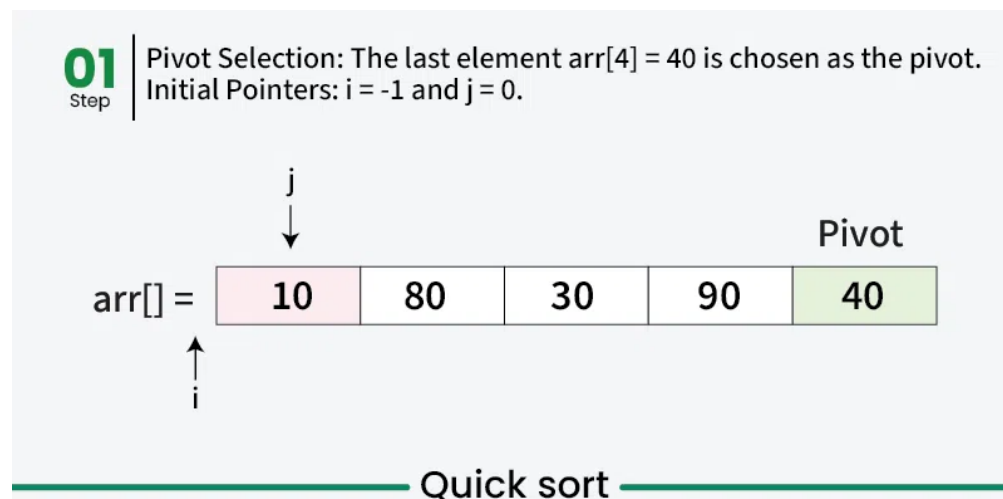


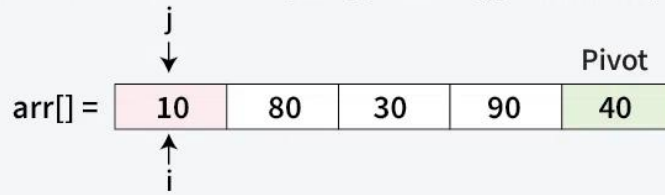
Figure 4 (geeksforgeeks.org)

Example of quicksort(geeksforgeeks.org):



02
Step

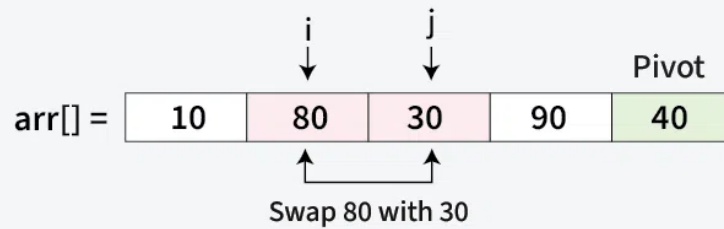
Since, $\text{arr}[j] < \text{pivot}$ ($10 < 40$)
Increment i to 0 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Quick sort

04
Step

Since, $\text{arr}[j] < \text{pivot}$ ($30 < 40$)
Increment i by 1 and swap $\text{arr}[i]$ with $\text{arr}[j]$. Increment j by 1



Quick sort

05
Step

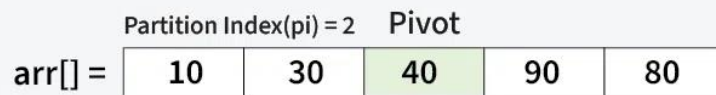
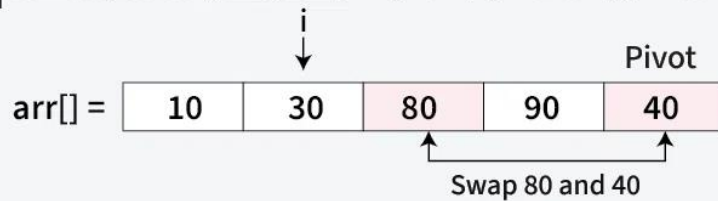
Since, $arr[j] > pivot$ ($90 < 40$)
No swap needed. Increment j by 1



Quick sort

06
Step

Since traversal of j has ended. Now move pivot to its correct position, Swap $arr[i + 1] = arr[2]$ with $arr[4] = 40$.



Quick sort

Example of Quicksort code output:

```
Microsoft Visual Studio Debug Console
Example array list using quick sort:

Unsorted array: 64 34 25 12 22 11 90

Sorting the example array:
Array after partitioning with pivot 90: 64 34 25 12 22 11 90
Array after partitioning with pivot 11: 11 34 25 12 22 64
Array after partitioning with pivot 64: 11 34 25 12 22 64
Array after partitioning with pivot 22: 11 12 22 34 25
Array after partitioning with pivot 25: 11 12 22 25 34
Sorted array: 11 12 22 25 34 64 90
```

Code output of array elements 10,000 to 40,000

```
First 10 elements of Unsorted array of 10000: 41 8467 6334 6500 9169 5724 1478 9358 6962 4464
First 10 elements of the sorted array of 10000: 0 0 2 3 3 3 4 4 6 8
First 10 elements of Unsorted array of 20000: 19484 2693 17194 10605 17059 6838 556 15179 18296 10782
First 10 elements of the sorted array of 20000: 0 0 1 2 2 2 6 6 7 7
First 10 elements of Unsorted array of 30000: 8970 8320 14252 20912 1360 6954 10443 11838 23376 652
First 10 elements of the sorted array of 30000: 1 1 3 3 3 4 4 5 5 6
First 10 elements of Unsorted array of 40000: 3154 19830 8543 16859 9519 8328 9072 19727 17818 5747
First 10 elements of the sorted array of 40000: 0 1 1 1 1 3 4 4 4 5

Quicksort Times:
for array of 10000 elements: 2 milliseconds
for array of 20000 elements: 4 milliseconds
for array of 30000 elements: 8 milliseconds
for array of 40000 elements: 9 milliseconds
Name: Adam Lee
Jagnumber: J00762612
```

Code for Quicksort:

```

1 // A program that takes 10000 to 40000 elements in each individual array and sorts them using quicksort algorithm
2 // credit - chegg.com
3 // credit - geeksforgeeks.com
4 // credit - stackoverflow.com
5 // Modified by:
6 // Name: Adam Lee
7 // JagNumber: J00762612
8 // Date: 10/31/2024
9
10
11 #include <iostream>
12 #include <stdio.h>
13 #include <cstdlib> // Include this for rand()
14 #include <chrono> // For measuring time
15
16 using namespace std;
17 using namespace std::chrono; // For high_resolution_clock
18
19 // Swap function
20 void swap(int* a, int* b)
21 {
22     int temp = *a;
23     *a = *b;
24     *b = temp;
25 }
26
27 // This function selects the last element as the pivot and places it in the right place
28 int partition(int array[], int min, int max)
29 {
30     int pivot = array[max]; // Select the pivot element
31     int index = (min - 1); // Store the index
32
33     for (int j = min; j <= max - 1; j++)

```

```

33     for (int j = min; j <= max - 1; j++)
34     {
35         // If current element is smaller than the pivot
36         if (array[j] < pivot)
37         {
38             index++;
39             swap(&array[index], &array[j]);
40         }
41     }
42     swap(&array[index + 1], &array[max]);
43
44     return (index + 1);
45 }
46
47 // Function to apply quick sort
48 void quickSort(int array[], int min, int max, bool isExample = false)
49 {
50     if (min < max)
51     {
52         int part = partition(array, min, max);
53
54         // Print the partition with pivot only for the example array
55         if (isExample) {
56             printf("Array after partitioning with pivot %d: ", array[part]);
57             for (int k = 0; k < max + 1; k++) {
58                 printf("%d ", array[k]);
59             }
60             printf("\n");
61         }
62
63         quickSort(array, min, part - 1, isExample);
64         quickSort(array, part + 1, max, isExample);
65     }

```

```

65     }
66 }
67
68 // Display array
69 void display(int array[], int size)
70 {
71     for (int i = 0; i < size; i++)
72         cout << array[i] << " ";
73     cout << endl;
74 }
75
76 // Main function
77 int main()
78 {
79     int array[] = { 64, 34, 25, 12, 22, 11, 90 }; // Sample array
80     int n = sizeof(array) / sizeof(array[0]); // Compute length
81     printf("Example array list using quick sort:\n\n");
82
83     // Display unsorted array
84     printf("Unsorted array: ");
85     display(array, n); // Display unsorted array
86     printf("\n");
87
88     printf("Sorting the example array:\n");
89     quickSort(array, 0, n - 1, true); // Set isExample to true to print the steps
90     printf("Sorted array: ");
91     display(array, n); // Display sorted array
92     printf("\n");
93
94     // Testing with sample random data
95     const int sizes[] = { 10000, 20000, 30000, 40000 };
96     long long times[4];
97

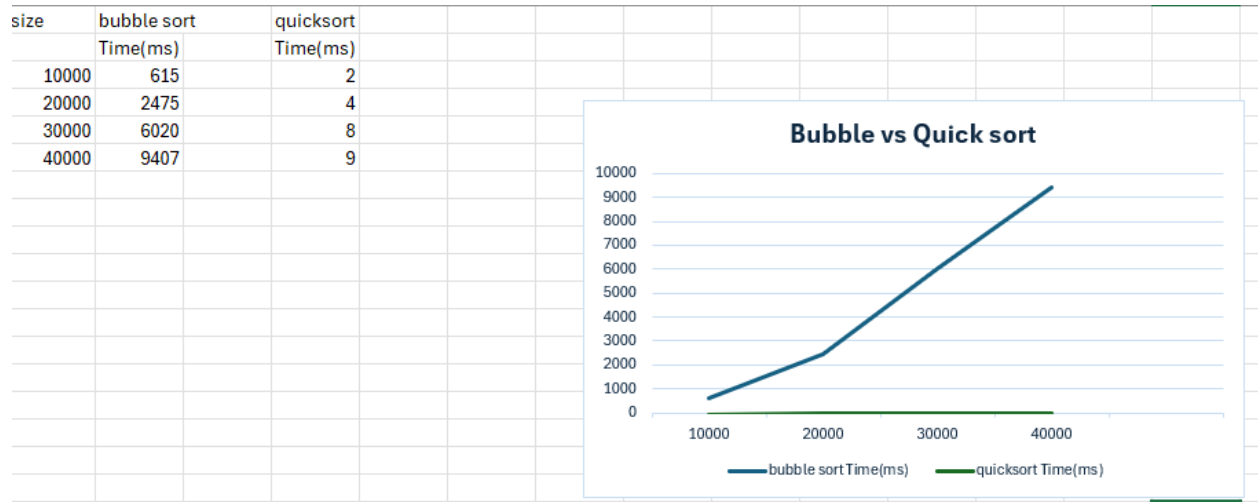
```

```

97
98     for (int i = 0; i < 4; ++i) {
99         int size = sizes[i];
100         int* array_random = new int[size];
101
102         // Fill the array with random numbers
103         for (int j = 0; j < size; j++)
104             array_random[j] = rand() % size;
105
106         printf("\nFirst 10 elements of Unsorted array of %d: ", size);
107         display(array_random, 10);
108
109         auto start = high_resolution_clock::now();
110         quickSort(array_random, 0, size - 1);
111         auto stop = high_resolution_clock::now();
112         times[i] = duration_cast<milliseconds>(stop - start).count();
113
114         printf("\nFirst 10 elements of the sorted array of %d: ", size);
115         display(array_random, 10);
116         delete[] array_random; // Clean up dynamically allocated memory
117     }
118
119     // Print the sorting times
120     printf("\nQuicksort Times:\n");
121     for (int i = 0; i < 4; ++i) {
122         printf("for array of %d elements: %lld milliseconds\n", sizes[i], times[i]);
123     }
124
125     printf("Name: Adam Lee\nJagnumber: J00762612\n");
126     return 0;
127 }
128
129

```

Excel spreadsheet to compare the two sorting algorithms.



Work cited

“Bubble Sort Algorithm.” *GeeksforGeeks*, 6 Oct. 2024, www.geeksforgeeks.org/bubble-sort-algorithm/. Accessed 29 Oct. 2024.

“Implementing a Timer in C++.” *Stack Overflow*, stackoverflow.com/questions/30702759/implementing-a-timer-in-c. Accessed 28 Oct. 2024.

“Quick Sort Algorithm.” *GeeksforGeeks*, 6 Oct. 2024, www.geeksforgeeks.org/quick-sort-algorithm/. Accessed 29 Oct. 2024.

“Question: I Only Need the Code for This Project, Not the Excel or PDF Files. Thank You.” *Chegg.Com*, 21 Mar. 2022, www.chegg.com/homework-help/questions-and-answers/need-code-project-excel-pdf-files-thank--q94435201. Accessed 25 Oct. 2024.